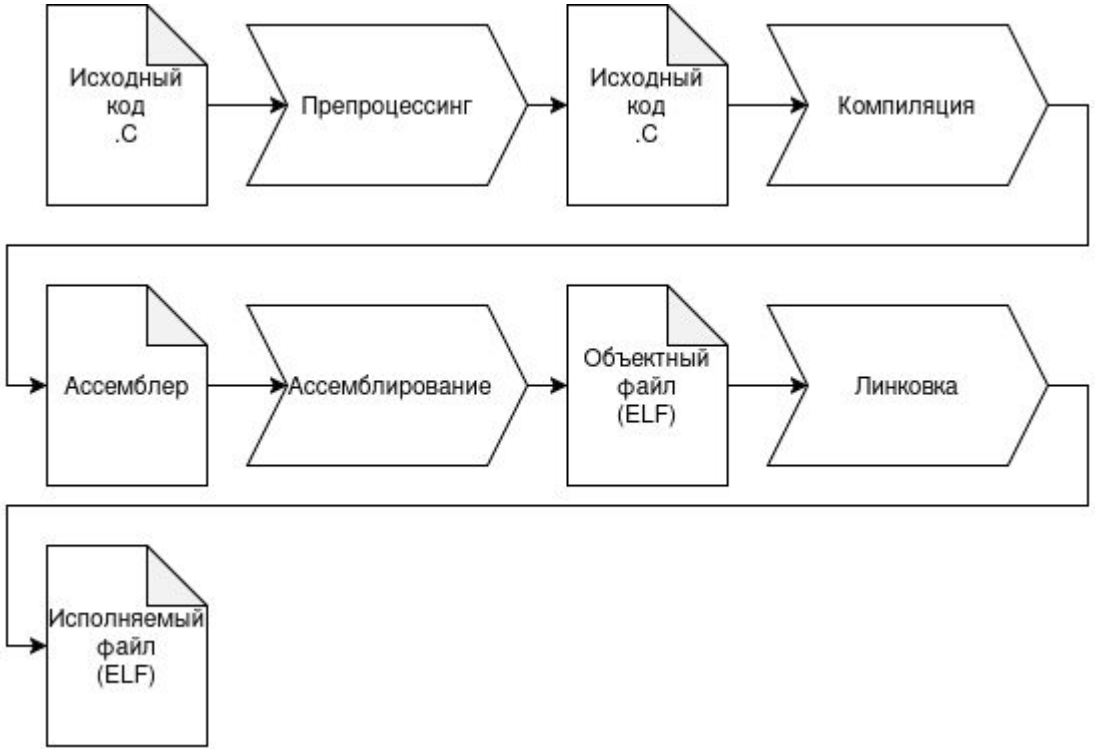
Основы языка C

Повторение: сборка программ

- Какие стадии проходит программа от исходного кода до запуска?
- Зачем нужна каждая стадия?
- Что является входом и выходом каждой стадии?

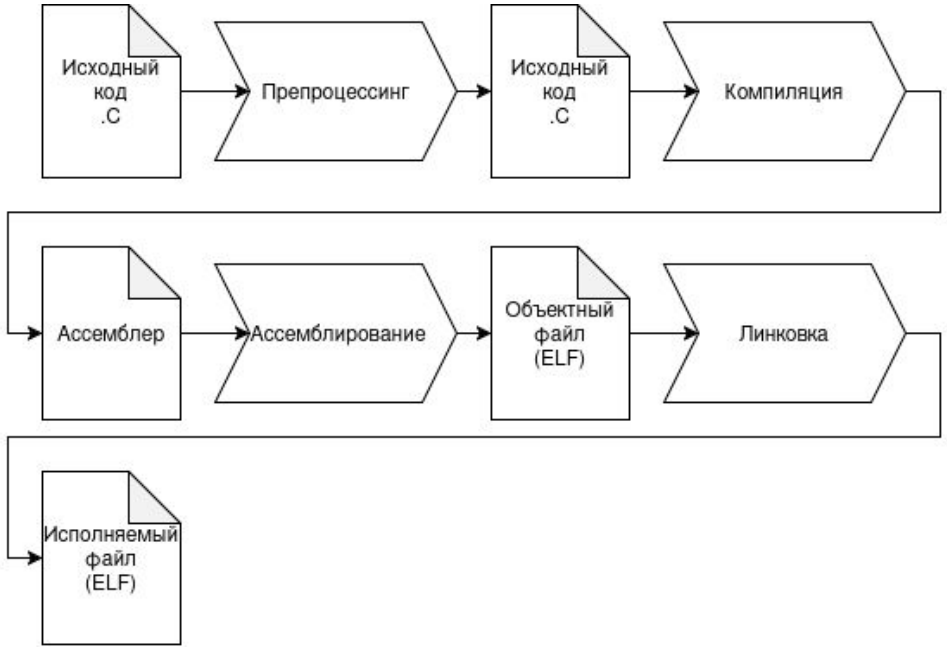
Этапы сборки

- **Препроцессинг**
  Директивы #include и #define.
  Исходный текст .c -> текст .i
- **Компиляция**
  Текст .i -> ассемблер .s
- **Ассемблирование**
  Ассемблер .s -> объектный .o
- **Линковка**
  Объекты и библиотеки -> программа



Драйвер компиляции

- gcc и clang принимают файлы и фла
  `gcc hello.c -o hello`
- Посмотреть команды драйвера:
  `gcc -### hello.c`
- Остановиться после стадии:
  -E (препроцессинг)
  -S (ассемблер)
  -c (компиляция)



Почему язык C

- Относительная “низкоуровневость” C помогает понять устройство программ и компьютера.
- Проблемы языка C были решены в других языках. Изучая C, мы поймем почему другие языки устроены так.
- Небольшое ядро языка позволяет сосредоточиться на алгоритмах.
- C все еще используется в промышленной разработке.
  Например: ядро Linux, PostgreSQL, CPython, SQLite, OpenSSL, KasperskyOS, Windows.
- Этот курс готовит к C++, ассемблеру и системному программированию



https://uguu.org/src_hazuki_cc.html



https://uguu.org/src_hazuki_cc.html

Стандарт языка C

- Стандарт задает синтаксис, семантику и стандартную библиотеку.
- Он нужен для переносимости программ и совместимости реализаций: программист и разработчик компилятора опираются на общие правила.
- В курсе используем C99. Флаг компилятора: -std=c99.
- Другие редакции: C11, C17, C23. Некоторые свойства зависят от реализации; для некоторых ситуаций стандарт не задает требований.

Стандарт языка C: Синтаксис и семантика

- Синтаксис и семантика – понятия, которые используются для описания любых языков (не только языков программирования)
- Стандарт языка содержит список синтаксических и семантических правил.
- Синтаксические правила определяют то, как должны быть написаны языковые конструкции.
- Семантические правила определяют то, что языковые конструкции означают.

Стандарт языка C: Синтаксис и семантика

- Синтаксические правила определяются грамматикой языка, их можно легко проверить и обнаружить синтаксические ошибки:

- `int a = 3; // синтаксически корректная конструкция`
- `int a = 3 + (2; // синтаксически некорректная конструкция`

Стандарт языка C: Синтаксис и семантика

- Семантические правила иногда проверить не так просто

```
int square(int x) {   // вызываемая функция должна быть где-то определена
    return x * x;
}

int main(void) {
    square("hello");  // функция должна быть вызвана с корректными аргументами
    return 0;
}
```

Простейшая программа

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- Ваша первая программа на C

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- Директива `include` буквально подставляет все содержимое указанного файла там, где она была вызвана
- `stdio.h` является частью библиотеки, обеспечивающей поддержку работы с файлами и предоставляющей функции ввода/вывода

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- Функция main является стандартной точкой входа в вашу программу
- Она должна быть ровно одна
- По соглашению при корректном завершении работы программы main возвращает 0

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- Функция `printf` отвечает за вывод
- Вызов функции `printf`: `printf (формат, аргументы)`

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
./hello
Hello, world!
```

https://godbolt.org/z/P31vjKv49

Минимум о функциях

- Объявление функции задает ее сигнатуру (т.е. определяет тип ее аргументов и возвращаемого значения):

```
int sum (int first, int second);
void foo ();
```

- Определение функции содержит ее тело:

```
int fibb (int n) {
    return fibb (n - 1) + fibb (n - 2);
}
```

- Объявлений функции может быть сколько угодно, главное, чтобы у них совпадала сигнатура. Определение должно быть ровно одно.

Типы данных

- Что такое тип данных?

Типы данных

- Что такое тип данных?
- Тип данных это множество всех его допустимых значений:

  Пример: 1073741823 помещается в `int`, а 8589934584 или 3.22 – нет
  (если считать, что размер `int` 32 бита, но это не всегда так)

- Тип данных это множество всех его допустимых операций:

  Пример: Операция + допустима как для типа `int`, так и для типа `double`.
  Операция `%` для `double` вообще не имеет смысла

Типы данных. Целочисленные типы

- `char`, `short`, `int`, `long`, `long long` – знаковые типы
  - `1 = sizeof(char) ≤ … ≤ sizeof(int) ≤ … ≤ sizeof(long long)`
  - Константа `CHAR_BIT` ≥ 8
  - Диапазон значений: от -2^{x-1} до $2^{x-1}-1$,

    где `x = sizeof(T) * CHAR_BIT`
- `unsigned char`, `unsigned short`, `unsigned int`, `unsigned long`, `unsigned long long` – беззнаковые типы
  - Диапазон от 0 до 2^x-1, где `x = sizeof(T) * CHAR_BIT`

- Обычно `sizeof(T) == sizeof(unsigned T)`

Типы данных. Целочисленные типы

- Иногда бывают необходимы типы с гарантированным размером:
  - `int32_t, int64_t`

- Их размер зафиксирован и не от чего не зависит
- Содержатся в `stdint.h`

Типы данных. Вещественные типы

- Для некоторых типов вычислений удобно использовать числа с плавающей точкой
  - `float, double`

Типы литералов

- Литерал – число, символ, например 1, 2, ‘a’, “hello”.
- Целый литерал без суффикса получает первый подходящий тип: `int, long, long long`.

Литерал	Тип	Литерал	Тип
`1`	`int`	`1.0`	`double`
`1u`	`unsigned`	`1.0f`	`float`
`1ll`	`long long`	`‘0’`	`char`
`1ull`	`unsigned long long`	`“hello”`	`const char[5]`

- Иногда литералы необходимы: 1 << 40 – UB (ошибка) – переполнение знакового типа. 1ULL << 40: левый операнд имеет тип `unsigned long long` (ULL это как минимум 64 бита).

Преобразования типов

- Преобразование типов – получение значения одного типа из значения другого по правилам языка.

- Оно нужно для смешанных вычислений, передачи аргументов и записи результата в переменную другого типа.

Явные преобразования типов

- Явное приведение задаем записью `(тип)выражение`.
- Пример: чтобы избежать целочисленного деления: `(double)a / b` дает вещественное частное двух целых.
- Преобразование может терять данные: `(int)3.9` дает `3`

Неявные преобразования типов

- Неявное преобразование выполняется автоматически.
  - Пример: `double x = 5;`

- При операциях с типами разного размера, выигрывает тип с большим размером:
  - `(unsigned char) 42 - 69 = -17 // uchar - int = int`
  - `‘0’ * 40 = 1920 // char * int = int`

- Смешивание знакового и беззнакового дает беззнаковый результат:
  - `5 - 10u = 4294967291u // int - unsinged = unsigned`

- Смешивание плавающего и целого дает плавающий результат:
  - `5.0 / 2 = 2.5 // double / int = double`

Операция, выражение, оператор

```
1.  int result = 0;
2.  result = 2 + 3 * 4;
3.  printf("%d\n", result);
```

- Что является оператором?
- Что является операцией?
- Что является выражением?

Операция, выражение, оператор

```
1.  int result = 0;
2.  result = 2 + 3 * 4;
3.  printf("%d\n", result);
```

- Операторы (statements): все, что заканчивается ;
- Операции (operators): `=`, `+`, `*`, `printf (...)` (оператор вызова функции)
- Выражения (expression): `2 + 3 * 4`, `result = 2 + 3 * 4`

https://en.cppreference.com/cpp/language/statements

Основные операции C

- Арифметические:
  `+, -, *, /, %, x++, x--`
- Сравнения:
  `<, <=, >, >=, ==, !=`
- Логика:
  `!, &&, ||`
  0 означает false, любое ненулевое значение – true.
- Присваивающие операции:
  `=, +=, -=, *=, /=, %=`

Битовые операции

- `&` И
- `|` ИЛИ
- `^` исключающее ИЛИ
- `~` инверсия
- `<<` и `>>` битовые сдвиги

```
unsigned int x = 10U; // ...1010
unsigned int y = 12U; // ...1100
x & y;                // 8
x | y;                // 14
x ^ y;                // 6
x << 1;               // 20
```

Приоритет и порядок вычислений

Приоритет	Операция
1	`[] () . -> x++ x--`
2	`sizeof(T) &x *x -x +x ~ ! ++x --x`
3	`* / %`
4	`+ -`
5	`<< >>`
6	`< > <= >=`
7	`== !=`

Приоритет	Операция
8	`&`
9	`^`
10	`|`
11	`&&`
12	`||`
13	`? :`
14	`= += -= *= /= %=`
15	`,`

Приоритет и порядок вычислений

1. `2 + 3 * 4`
2. `(2 + 3) * 4`
3. `x & 1 == 0`
4. `(x & 1) == 0`

Приоритет и порядок вычислений

1. `2 + 3 * 4 = 14`
2. `(2 + 3) * 4 = 20`
3. `x & 1 == 0 = false`
4. `(x & 1) == 0 = true`

Короткая (ленивая) логика: условие if

```
int a = 10;
int b = 0;

if (b != 0 && a / b > 2) {
    puts("Quotient > 2");
} else {
    puts("Condition is false");
}
```

- && сначала вычисляет левый операнд.
- Если b == 0, левый операнд ложен: деление не выполняется.
- Если b != 0, вычисляется a / b > 2.
- Для || правый операнд не вычисляется, если левый уже истинен.

Поведение программы

- Стандарт языка Си точно определяет наблюдаемое поведение программ на языке Си, за исключением следующих случаев:
- Все синтаксически некорректные программы
- Среди синтаксически корректных программ:
  - Неопределенное поведение (undefined behaviour, UB, УБ)
  - Неуточненное поведение (unspecified behaviour)
  - Поведение, определяемое реализацией (implementation-defined behaviour)

Неопределенное поведение

- Undefined behavior: стандарт не предъявляет требований к поведению программы в такой ситуации и ничего не гарантирует.
- При UB может произойти что угодно: аварийное завершение, неверный результат или корректное (ожидаемое) выполнение кода.
- Компилятор не обязан диагностировать UB.
- Компилятор оптимизирует, исходя из того, что на корректном пути выполнения UB не возникает.
- Примеры: деление целого на ноль, знаковое переполнение, выход за границы массива, обращение к неинициализированной переменной.

https://en.cppreference.com/c/language/behavior

Неопределенное поведение

```
int greater(int x) {
    return x + 1 > x;
}
```

- Если x + 1 представимо в int, результат сравнения равен 1.
- При x == INT_MAX сложение вызывает UB – знаковое переполнение
- Clang 18.1 при -O2 заменяет функцию на return 1.
  Если мы передадим x == INT_MAX, получим 1.

https://godbolt.org/z/e46dh4159

Неопределенное поведение

```
#include <stdio.h>

int main() {
  unsigned int x = 0;
  while (x >= 0) {
    x = x + 1;
  }
}

void unreachable_function() {
  printf("You would never see this!\n");
}
```

- Цикл не завершится:
  x – беззнаковый.
- У цикла нет наблюдаемых побочных эффектов. Это UB.
- Что покажет программа при -O0 и -Ofast?

https://godbolt.org/z/x3G3aYrEx

Неуточненное поведение

```
foo() + bar() + baz()
```

- Unspecified behaviour: существует несколько (как минимум два) разрешенных сценариев поведения программы в этой ситуации. Каждое поведение при этом может приводить к одному из допустимых результатов.

- В данном примере порядок вызовов функций не определен

Какое-то поведение…

```
i = 1;
i = ++i + i++;
```

- Что по-вашему здесь произойдет?
- Чему будет равно `i`?

https://godbolt.org/z/xYnfj5eMj

Поведение, определяемое реализацией

- Implementation-defined behavior (IDB): стандарт допускает несколько вариантов и требует документировать выбор реализации.
    - Пример: количество бит в байте

```
int char_is_signed(void) {
    char c = (char)-1;
    return c < 0;
}
```

- GCC 14.1, -fsigned-char: вывод 1
- GCC 14.1, -funsigned-char: вывод 0

- Различаться могут компилятор, платформа и флаги сборки.

- В этих двух конфигурациях программа корректна.

https://godbolt.org/z/sdj39YxGz

Литература: язык C

- Б. Керниган, Д. Ритчи. «Язык программирования C», 2-е издание.
- C. Прата. C Primer Plus
- ISO C99 + TC1–TC3, проект WG14 N1256 (link).
  Будет доступен на контрольных как единственный справочник, нужно научиться с ним работать.
- Справочник по языку C: https://en.cppreference.com/c/language

Литература: алгоритмы

- А. Бхаргава. «Грокаем алгоритмы».
- Т. Рафгарден. Algorithms Illuminated, части 1–4.
- Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. «Алгоритмы. Построение и анализ».

Extra: Vocabulary

- На вход вашей программе подается текст. Необходимо посчитать количество вхождений каждого уникального слова в данном тексте. Словом считается любая последовательность [A-Za-z0-9]+. Слова, различающиеся только регистром (верхний, нижний) считаем одинаковыми.
- На выходе ваша программа должна вывести каждое уникальное слово и количество его вхождений. Порядок вывода определяется порядком первого вхождения слова в текст.

- Пример входных данных:

```
C99-pointer, MEMORY!
123 3D null-pointer
```

- Пример выходных данных:

```
c99 1
pointer 2
memory
123 1
3d 1
null 1
```

Extra: Stack

- Необходимо реализовать стек.
- На вход вашей программе подается число n, после этого подается n последовательных операций. Операции могут быть двух видов:
  - “+ a”, число a помещается в тип int. Операция “+” означает положить значение в стек.
  - “-”. Операция означает достать значение из стека.
- На выходе программа должна вывести окончательное состояние стека, либо слово ERROR в случае ошибочной последовательности на входе

- Пример входных данных:

```
4
+ 42
+ 69
-
+ 322
```

- Пример выходных данных:

```
42
322
```